PRÁTICA ORIENTADA 02

Programação Web I
Prof. Dr. Renato William Rodrigues de Souza
IFCE Campus Boa Viagem
Curso: Análise e Desenvolvimento de Sistemas

Aluno: Alissa Garcia Moreira

Engenharia Reversa e Análise de Design de Software: Tropycaly

Boa Viagem - 2026

1. Compreensão do Sistema

1.1 Qual o objetivo do Sistema?

O sistema tem como principal objetivo permitir que os clientes realizem pedidos de uma pizzaria física especializada na venda de pizzas, lanches, esfihas, bebidas e combos. Além de facilitar a escolha dos produtos, o sistema também busca agilizar o processo de solicitação de entrega, reunindo em um único ambiente todas as etapas do pedido, desde a seleção dos itens até a definição da forma de pagamento e do endereço de entrega.

1.2 Quais são suas principais funcionalidades?

O sistema oferece diversas funcionalidades voltadas à praticidade do cliente. Entre as principais, destacam-se: Exibição de um cardápio virtual contendo todos os produtos disponíveis; Organização dos itens por categorias, combos e promoções; Navegação entre produtos semelhantes; Seleção de produtos e escolha da quantidade desejada; Inclusão de adicionais e observações específicas para cada item; Simulação de um carrinho de compras, permitindo visualizar os itens selecionados; Cálculo automático do valor total da compra; Escolha da forma de pagamento; Definição do endereço para entrega; Finalização do pedido.

Embora o cliente possa selecionar a forma de pagamento, o pagamento não é processado diretamente pelo sistema, sendo realizado posteriormente no momento da entrega ou retirada.

1.3 Como o usuário interage com o sistema?

A interação do usuário ocorre por meio de uma interface simples e intuitiva, composta por botões, imagens e campos de entrada. Inicialmente, o cliente navega pelo cardápio e escolhe a categoria desejada, como pizzas, sanduíches, bebidas ou promoções. Em seguida, seleciona o produto, informa a quantidade e utiliza o botão “Adicionar ao Carrinho” para inserir o item no pedido.
Durante todo o processo, o usuário pode visualizar os produtos adicionados, alterar quantidades, inserir observações e acompanhar o valor total da compra em tempo real. Ao concluir a seleção dos itens, o cliente utiliza a opção “Finalizar Pedido”, momento em que informa a forma de pagamento e o endereço de entrega..

1.4 Como os produtos estão organizados?

Os produtos são organizados de forma responsiva, adaptando-se ao tipo de dispositivo utilizado pelo cliente. Na versão mobile, os produtos são exibidos em uma única coluna, facilitando a navegação em telas menores. Já na versão

desktop, os itens são distribuídos em duas colunas, proporcionando melhor aproveitamento do espaço e tornando a visualização mais organizada.

Além disso, o sistema possui uma barra de navegação principal dividida em três seções: categorias, combos e promoções. Dentro de cada seção, os produtos são agrupados por tipo.

2. Análise de Arquitetura

2.1 Tipo de Arquitetura?

O sistema analisado apresenta uma arquitetura baseada no modelo cliente-servidor, com indícios de organização em camadas, ainda que de forma não totalmente explícita. Nesse tipo de arquitetura, há uma separação entre a interface do usuário, o processamento das requisições e o armazenamento dos dados, o que contribui para uma melhor estruturação do sistema e facilita sua manutenção.
A camada de front-end é responsável pela interação com o usuário, sendo desenvolvida com tecnologias como HTML5, CSS e JavaScript, além do uso de bibliotecas e frameworks como Bootstrap, jQuery, Modernizr e Popper. Essa camada permite a visualização do cardápio, a navegação entre categorias, combos e promoções, bem como a seleção de produtos e interação com o carrinho de compras.
A camada de back-end utiliza a linguagem PHP, que é responsável por processar as requisições enviadas pelo cliente, aplicar as regras de negócio, como cálculo do valor total do pedido e organização dos itens selecionados, e retornar as respostas adequadas ao usuário. Já a camada de dados, embora não detalhada explicitamente, é responsável pelo armazenamento das informações do sistema, como produtos e pedidos, sendo acessada pelo back-end.
O sistema utiliza o servidor web Apache e está hospedado na plataforma HostGator, caracterizando uma infraestrutura tradicional de hospedagem.
De modo geral, a arquitetura adotada é simples e funcional, sendo adequada para aplicações de pequeno a médio porte, como um sistema de cardápio digital com pedidos online.

3. Análise de design

3.1 Coesão, acoplamento e separação de responsabilidades.

O sistema não apresenta uma aplicação clara e bem estruturada dos princípios de coesão, acoplamento e separação de responsabilidades. Observa-se que há uma tendência de centralização de funcionalidades, onde diferentes responsabilidades podem estar concentradas em um mesmo módulo ou arquivo, especialmente no back-end em PHP e em scripts JavaScript no front-end. Isso indica uma baixa coesão, pois elementos que deveriam ter

funções específicas acabam desempenhando múltiplos papéis dentro do sistema.

Em relação ao acoplamento, é provável que ele seja relativamente alto, uma vez que as partes do sistema parecem depender diretamente umas das outras, sem a presença de camadas bem definidas ou uso de padrões arquiteturais mais robustos. Por exemplo, a interface pode estar diretamente ligada às regras de negócio e à manipulação de dados, dificultando alterações isoladas sem impactar outras partes do sistema.

Quanto à separação de responsabilidades, embora exista uma divisão básica entre front-end e back-end, essa separação não é totalmente estruturada. Não há evidências claras de padrões como MVC (Model-View-Controller) ou outras abordagens que promovam uma divisão mais organizada entre apresentação, lógica de negócio e dados. Isso pode tornar o sistema mais difícil de manter, evoluir e escalar ao longo do tempo.

4. Padrões de Projeto

4.1 O sistema aparenta utilizar padrões?

O sistema não apresenta evidências claras da utilização explícita de padrões de projeto formais. A aplicação parece ter sido desenvolvida de maneira mais direta, utilizando PHP no back-end e JavaScript com jQuery no front-end, sem a adoção estruturada de padrões como MVC, Factory ou Singleton. Apesar disso, podem existir implementações implícitas ou não padronizadas de algumas dessas ideias, mas sem uma organização clara que caracterize o uso intencional de padrões de projeto.

4.2 Onde poderiam existir Factory, SIngleton e MVC?

O padrão Factory poderia existir na criação de objetos relacionados ao domínio do sistema, como produtos (pizza, sanduíche, bebida) e pedidos, centralizando essa responsabilidade em uma única classe. O Singleton poderia ser aplicado na gestão da conexão com o banco de dados, garantindo que apenas uma instância seja utilizada durante a execução do sistema. Já o padrão MVC poderia existir na estrutura geral da aplicação, separando as responsabilidades entre Model (dados como produtos e pedidos), View (interface do usuário, como o cardápio) e Controller (lógica que processa as ações do usuário).

4.3 Onde poderiam ser aplicados?

Esses padrões poderiam ser aplicados para melhorar a organização e manutenção do sistema. O Factory poderia ser implementado na criação de objetos no back-end, evitando repetição de código. O Singleton poderia ser

utilizado para gerenciar conexões com banco de dados ou configurações globais do sistema. O MVC poderia ser adotado para estruturar melhor o sistema, separando a lógica de negócio, a interface e o controle das requisições, tornando o código mais organizado, reutilizável e escalável.

5. Comparação com Sistema Didático

Critério	**Sistema Real (Pizzaria)**	**Sistema Didático (Acadêmico)**
Arquitetura	Cliente-servidor, com separação básica entre front e back	Monolítica, sem divisão clara de camadas
Coesão	Baixa, com responsabilidades pouco definidas	Baixa, com funções concentrando múltiplas tarefas
Acoplamento	Moderado a alto, com dependência entre partes do sistema	Alto, com forte dependência entre componentes
Organização	Limitada, sem padrão arquitetural bem definido	Pouca organização estrutural
Flexibilidade	Baixa, com dificuldade de adaptação e evolução	Muito baixa, difícil de modificar e expandir

6. Modelagem do Sistema.

6.1 Identificação de Entidades

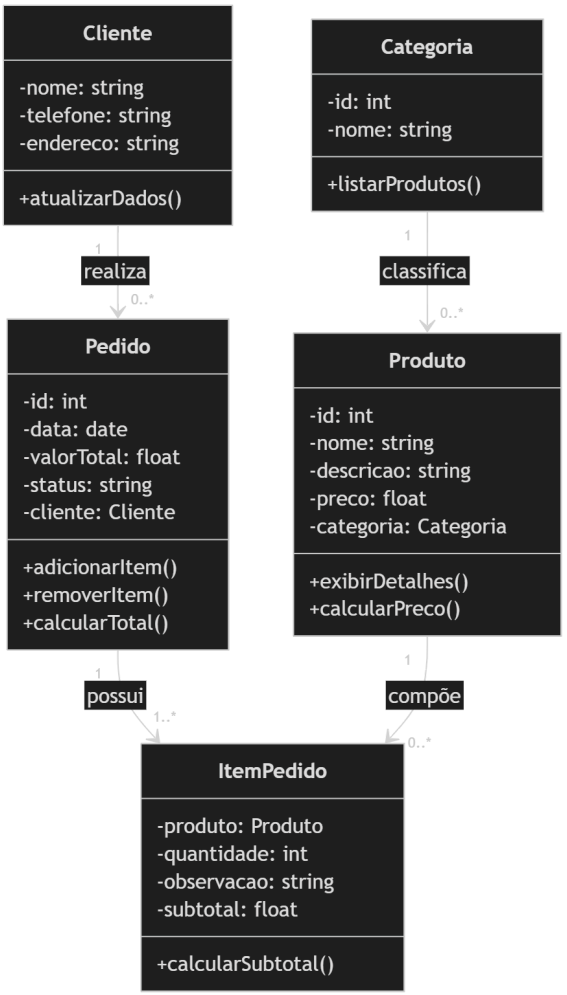
As principais entidades do sistema estão relacionadas ao domínio de pedidos de uma pizzaria. Entre elas, destacam-se Produto, Pedido, ItemPedido, Categoria e Cliente. A entidade Produto representa os itens disponíveis para venda, como pizzas, lanches e bebidas. A entidade Categoria organiza os produtos em grupos, como combos, promoções ou tipos de alimentos. Pedido representa a compra realizada pelo cliente, contendo informações gerais da solicitação. ItemPedido é responsável por relacionar os produtos ao pedido, incluindo quantidade e possíveis observações. Já a entidade Cliente pode armazenar informações como nome, endereço e contato, necessários para a entrega do pedido.

6.2 Definição de Classes

A classe Produto pode possuir atributos como id, nome, descrição, preço e categoria, além de métodos como exibirDetalhes() e calcularPreco(). A classe Categoria pode conter atributos como id e nome, com métodos simples como listarProdutos(). A classe Pedido pode ter atributos como id, data, valorTotal, status e cliente, e métodos como adicionarItem(), removerItem() e calcularTotal(). A classe ItemPedido pode possuir atributos como produto, quantidade, observação e subtotal, além de métodos como calcularSubtotal().
A classe Cliente pode conter atributos como nome, telefone e endereço, com métodos como atualizarDados(). De forma geral, essas classes representam a estrutura básica do sistema e permitem organizar melhor os dados e comportamentos envolvidos no processo de compra..

6.3 Diagrama de Classes:

- Classes
- Relacionamentos
- Multiplicidade



6.4 Justificativa das Escolhas:

As classes definidas para o sistema foram escolhidas com base na análise do comportamento visível da aplicação, mesmo sem acesso ao seu backend. A interface do sistema permite identificar claramente um fluxo de pedidos típico de uma pizzaria, o que possibilita inferir as principais entidades envolvidas. A classe Produto foi definida por representar os itens exibidos no cardápio, como pizzas, hambúrgueres e bebidas, sendo um elemento central do sistema. Já a classe Categoria foi incluída para organizar esses produtos, conforme observado na separação por grupos como "Produtos", "Combos" e "Promoções", melhorando a estrutura e navegação. A classe Pedido foi identificada a partir da presença do carrinho de compras e do processo de finalização, indicando que o sistema registra as solicitações realizadas pelos clientes. Para detalhar os itens dentro de um pedido, foi criada a classe ItemPedido, responsável por armazenar informações como produto selecionado, quantidade e observações, refletindo o funcionamento típico de sistemas de pedidos. A classe Cliente foi considerada necessária devido à existência de informações como contato e endereço para entrega, visíveis na proposta do sistema, mesmo que não diretamente exibidas na interface inicial.

7. Problemas Identificados:

7.1 Limitações de arquitetura

O sistema apresenta limitações relacionadas à sua arquitetura, que aparenta ser simples e pouco estruturada. Não há indícios claros de uma separação bem definida em camadas ou da utilização de padrões arquiteturais como MVC. Além disso, o uso de tecnologias mais tradicionais, como PHP sem framework e bibliotecas JavaScript desatualizadas, indica uma arquitetura mais monolítica e menos preparada para escalabilidade. Isso dificulta a evolução do sistema, especialmente caso seja necessário adicionar novas funcionalidades, como integração com sistemas de pagamento ou aplicativos móveis.

7.2 Alto acoplamento

O sistema demonstra um nível elevado de acoplamento entre seus componentes, principalmente entre a interface (front-end) e a lógica de negócio. É provável que scripts JavaScript estejam diretamente ligados à manipulação de dados e que o PHP misture regras de negócio com apresentação. Esse alto acoplamento faz com que alterações em uma parte do sistema impactem diretamente outras, reduzindo a flexibilidade e aumentando o risco de erros durante modificações.

7.3 Dificuldade de manutenção

Devido à ausência de uma estrutura bem organizada, à falta de separação clara de responsabilidades e ao alto acoplamento, o sistema tende a apresentar dificuldades de manutenção. A leitura e compreensão do código tornam-se mais complexas, principalmente para novos desenvolvedores. Além disso, a atualização de funcionalidades ou correção de erros pode exigir alterações em múltiplos pontos do sistema, aumentando o tempo de desenvolvimento e a possibilidade de introdução de novos problemas.

8. Proposta de Arquitetura

8.1 Organização de camadas ou MVC:

Como proposta de melhoria, recomenda-se a adoção do padrão MVC (Model-View-Controller) para organizar melhor a estrutura do sistema. Nesse modelo, o Model seria responsável pelos dados e regras de negócio, como Produto, Pedido e Cliente. A View ficaria encarregada da interface com o usuário, exibindo o cardápio, carrinho e telas de finalização. Já o Controller atuaria como intermediador, recebendo as requisições do usuário, processando as ações (como adicionar itens ao pedido) e retornando as respostas adequadas. Essa organização em camadas contribui para um código mais limpo, reutilizável e de fácil manutenção.

8.2 Separação de responsabilidades:

A separação de responsabilidades deve ser aplicada de forma clara entre as partes do sistema. Cada classe ou módulo deve ter uma única responsabilidade bem definida. Por exemplo, classes de modelo devem apenas representar dados e regras de negócio, enquanto controladores devem gerenciar o fluxo da aplicação e as views devem cuidar apenas da apresentação. Além disso, a lógica de acesso a dados pode ser isolada em componentes específicos, como repositórios ou serviços, evitando que o código fique misturado e difícil de manter.

8.3 Componentes principais:

Na nova arquitetura proposta, o sistema pode ser dividido em componentes principais bem definidos. Entre eles, destacam-se o módulo de gerenciamento de produtos, responsável por cadastrar, listar e organizar os itens do cardápio; o módulo de pedidos, responsável por criar, atualizar e finalizar pedidos; o módulo de clientes, que gerencia dados como nome, telefone e endereço; e o módulo de pagamento, que pode futuramente integrar formas de pagamento

online. Além disso, há o componente de interface (front-end), responsável pela interação com o usuário, e o componente de back-end, que processa as regras de negócio e realiza a comunicação com o banco de dados. Essa divisão torna o sistema mais organizado, escalável e preparado para futuras melhorias.

9. Aplicação de Padrões

9.1 Como aplicar o Factory

O padrão Factory pode ser aplicado no sistema para centralizar a criação de objetos relacionados ao domínio, como Produto, Pedido e ItemPedido. Em vez de instanciar esses objetos diretamente em diferentes partes do código, seria criada uma classe responsável por essa criação, como uma "ProdutoFactory" ou "PedidoFactory". Por exemplo, ao criar um produto, a factory poderia receber o tipo (pizza, sanduíche, bebida) e retornar o objeto já configurado corretamente. Isso evita repetição de código, facilita a manutenção e permite adicionar novos tipos de produtos sem alterar várias partes do sistema. Além disso, a lógica de criação fica isolada em um único local, tornando o sistema mais organizado.

9.2 Como aplicar o Singleton

O padrão Singleton pode ser aplicado para gerenciar recursos que devem ter apenas uma única instância durante a execução do sistema, como a conexão com o banco de dados. Nesse caso, seria criada uma classe de conexão que garante que apenas um objeto seja instanciado. Sempre que o sistema precisar acessar o banco, ele utilizaria essa mesma instância. Isso melhora o controle de recursos, evita múltiplas conexões desnecessárias e pode contribuir para o desempenho da aplicação. Além disso, o Singleton também pode ser utilizado para gerenciar configurações globais do sistema, como parâmetros de ambiente ou configurações gerais da aplicação.

10. Reflexão Crítica

10.1 É possível modelar um sistema sem ver o código?

Sim, é possível modelar um sistema mesmo sem ter acesso ao código-fonte. A modelagem pode ser realizada com base na observação do funcionamento do sistema, na análise da interface, nas funcionalidades disponíveis e no comportamento esperado. A partir disso, é possível identificar entidades, fluxos, regras de negócio e interações entre os componentes. No entanto, essa modelagem tende a ser mais abstrata e pode não refletir exatamente a

implementação real, já que detalhes internos e decisões técnicas não ficam visíveis.

10.2 Qual a importância da modelagem?

A modelagem é fundamental no desenvolvimento de sistemas, pois permite compreender, organizar e representar a estrutura e o funcionamento da aplicação antes ou durante sua implementação. Ela auxilia na identificação de entidades, relacionamentos e responsabilidades, além de facilitar a comunicação entre desenvolvedores e outros envolvidos no projeto. Uma boa modelagem contribui para a redução de erros, melhora a qualidade do sistema e torna o desenvolvimento mais eficiente, além de servir como base para manutenção e evolução futura.

10.3 Qual a diferença entre sistema real e didático?

O sistema real foi desenvolvido e pensado para atender necessidades práticas, envolvendo usuários reais, requisitos, regras de negócio mais estabelecidas integrações externas, segurança, desempenho e escalabilidade. Já o sistema didático foi criado com fins educacionais, focando na compreensão de conceitos e técnicas de desenvolvimento, com menor complexidade e sem a necessidade de atender todos os requisitos de um ambiente real. O sistema real jpa aplica uma arquitetura mais consolidada de cliente-servidor. Já o sistema didático é um monolítico.

Link: https://github.com/AlissaGarcia/PWEB/tree/main/Analise-sistema